Cerca no informada

Bernat Orellana – PROP 2021/22

Graf

- Podem utilitzar grafs per anar representant/emmagatzemant l'arbre d'exploració de l'espai de solucions.
- Podem optar per una implementació pròpia

```
public class Node {
    public String name;
    public List<Edge> connections;
}
```

```
public class Edge {
    public Node start;
    public Node end;
    public double weight;
}
```

```
public class Graph {
    List<Node> nodes;
}
```

- Generalment és recomanable usar una llibreria externa (Jung)
  - http://jung.sourceforge.net/doc/index.html

Algorismes de cerca cecs

- Amplada

```
def bfs(graph, start):
    LNT = []
    LNO = [start]
    solucio = False
    while len(LNO) > 0 and not solucio:
        vertex = LNO.pop(0)
        if vertex not in LNT:
            if not  isSolucio(vertex):
                LNT.add(vertex)
                LF = graph[vertex]
                LNO.extend(LF - LNT)
            else:
                solucio = True
    tractar_solucio(vertex)
```

Algorismes de cerca cecs

- Profunditat i **IDS**

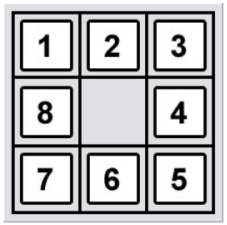
```
def dfs(graph, start):
    LNT = []
    LNO = [start]
    solucio = False
    while len(LNO) > 0 and not solucio:
        vertex = stack.pop()
        if vertex not in LNT:
            if not  isSolucio(vertex):
                LNT.add(vertex)
                LF = graph[vertex]
                LNO.expand(LF - LNT)
            else:
                solucio = True
    tractar_solucio(vertex)
```

Exercici

- Es proposa resoldre el problema de nine-puzzle amb una cerca en amplada, en profunditat i IDS.
- Es proporciona un projecte que incorpora:
  - Un esquelet de classes per representar la lògica del NinePuzzle i els seus moviments (Dir)
  - El projecte ja duu la llibreria Jung
  - La funció de visualització del graf i de la solució ja està implementada.



Initial State



Goal State

Objectiu

- Cal que implementeu a la classe **Main** les funcions:
  - cercaNoInformadaBFS
  - cercaNoInformadaDFS
  - cercaNoInformadaIDS
- Han de retornar l'estat del graf d'exploració en el moment de trobar la solució i el node solució. Retornen **null** si no es troba la solució.

Consideracions

- Al llarg de la exploració **és crucial** evitar cicles (és a dir, tornar a caure en una configuració de tauler que anteriorment ja hem fet)
- Hi ha dues opcions per tallar cicles:
  - Usar recorregut cap a dalt pel graf des del node actual fins a l'arrel de l'arbre per evitar moviments cíclics.
  - Usar LNT, excloent els nodes ja processats (veure pseudocodi dels algorismes)
  - Quines avantatges/inconvenients té cadascuna de les dues opcions ?
- Useu la funció *comparaEstat()* per comparar dos estats de **NinePuzzle**

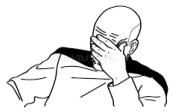
Consideracions

- Valoreu quina és la millor classe per contenir la LNO i la LNT. Recordeu que els nodes són de tipus ***NinePuzzle***:
  - ArrayList<NinePuzzle>
  - LinkedList<NinePuzzle>
  - Stack<NinePuzzle>
  - Queue<NinePuzzle>

- Feu un recompte de nodes explorats en cada cas i mostreu-lo a la sortida.

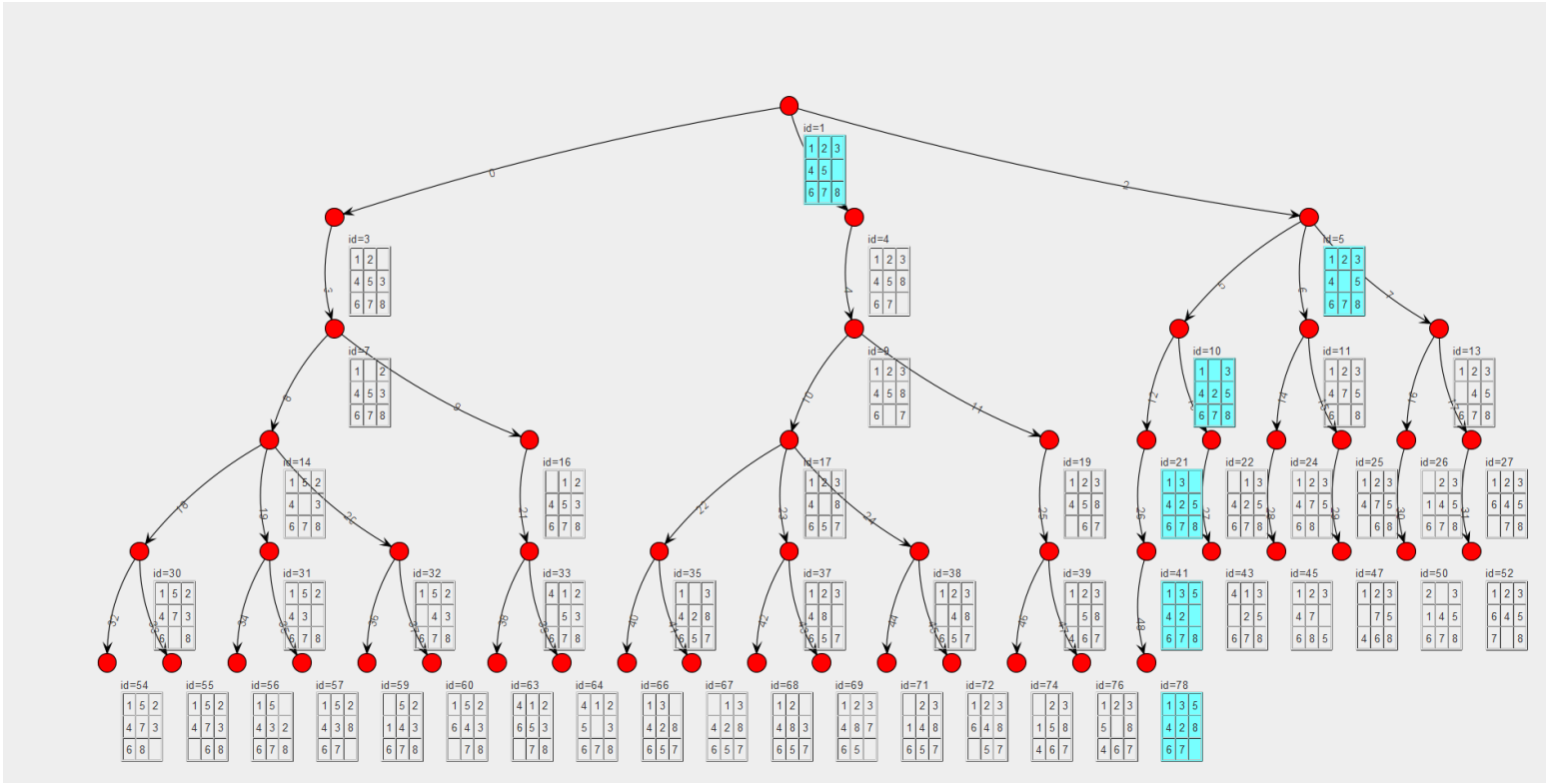
Consideracions

- Cerca en profunditat (DFS)
  - Programeu-la primer sense limitar la profunditat màxima i compareu la solució que ens dona respecte la cerca en amplada.
  - Assegureu-vos que no "acumuleu" els nodes ja recorreguts i que no estan a la LNO. Sinó, en memòria tindrem el mateix cost que una BFS.

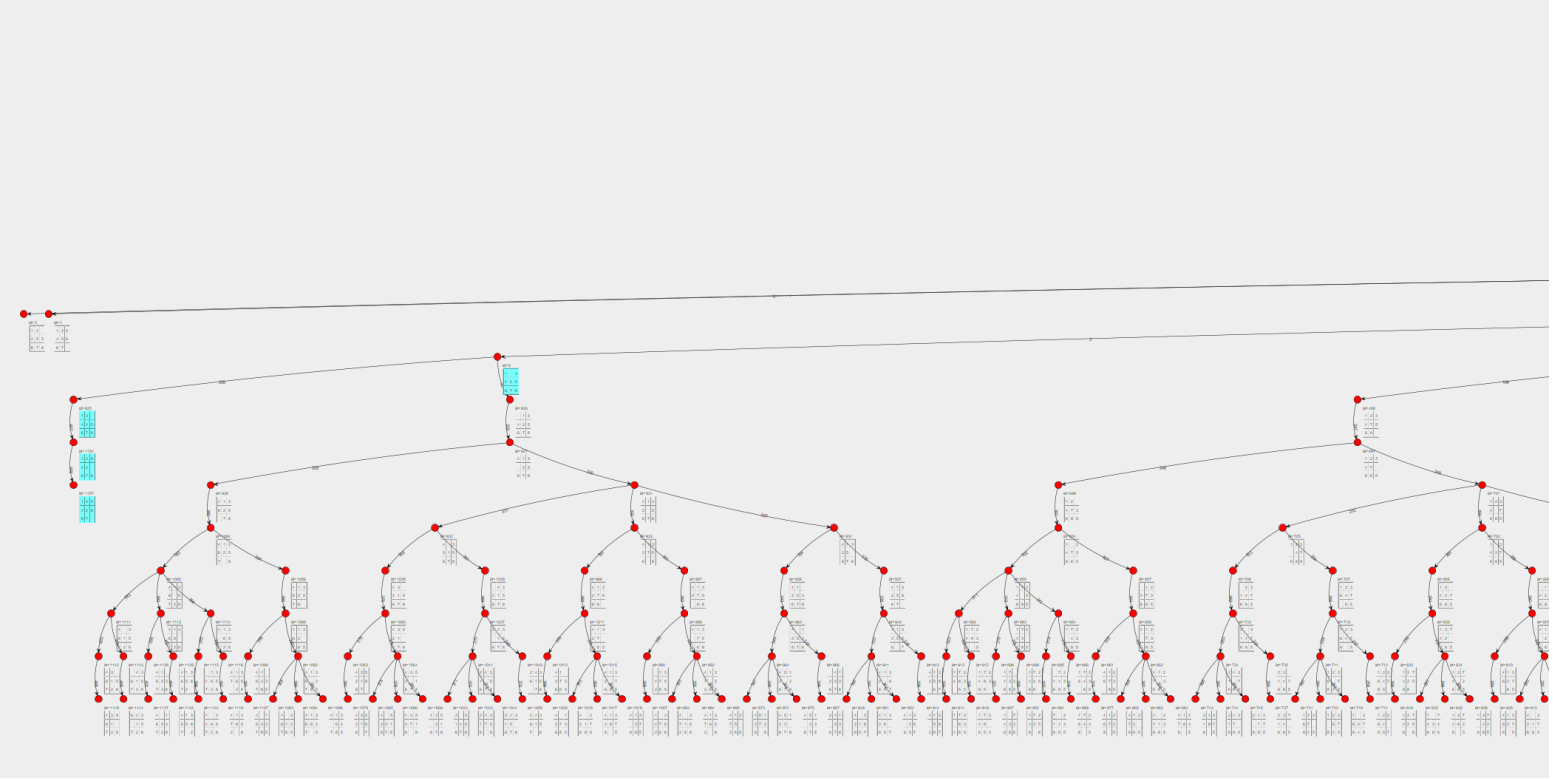


  - Proveu a implementar DFS usant una crida recursiva.
  - Limiteu la profunditat de cerca i feu el IDS. Verifiqueu el canvi de les solucions trobades.

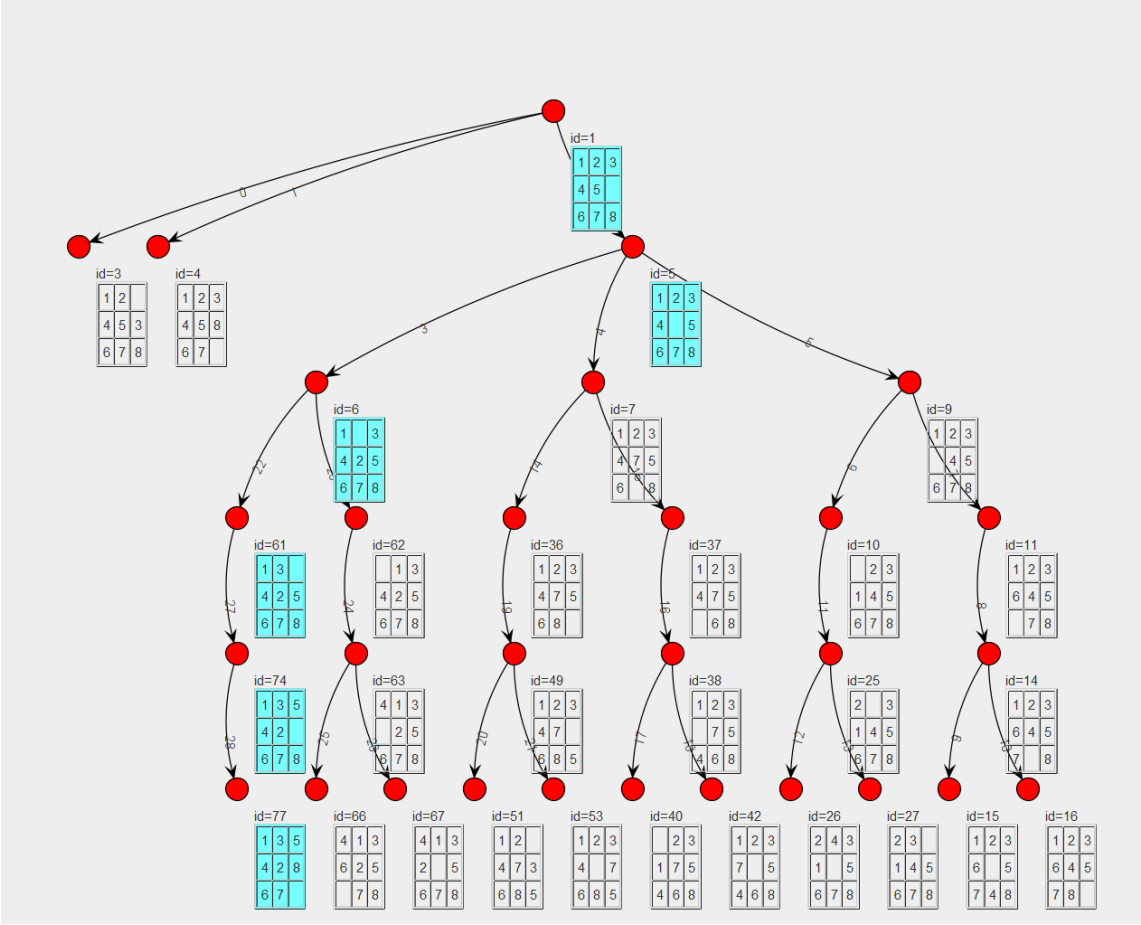
BFS


id=1
id=3
id=4
id=5
id=7
id=9
id=10
id=11
id=13
id=14
id=16
id=17
id=19
id=21
id=22
id=24
id=25
id=26
id=27
id=30
id=31
id=32
id=33
id=35
id=37
id=38
id=39
id=41
id=43
id=45
id=47
id=50
id=52
id=54
id=55
id=56
id=57
id=59
id=60
id=63
id=64
id=66
id=67
id=68
id=69
id=71
id=72
id=74
id=76
id=78

DFS (limitada a profunditat 10)



IDS


id=1
id=3
id=4
id=5
id=6
id=7
id=9
id=61
id=62
id=36
id=37
id=10
id=11
id=74
id=63
id=49
id=38
id=25
id=14
id=77
id=66
id=67
id=51
id=53
id=40
id=42
id=26
id=27
id=15
id=16